EVENT CONFLICT CHECKER

Course Project Submission for CSE1021: Introduction to Problem solving and programming

Submitted by: Aditee Singh

VIT Bhopal University,
Date: November 24, 2025

INTRODUCTION

Events conflicts are a common problem for students and working professionals that have to engage in and manage various activities. Struggling to organize overlapping events can lead to missed classes, double bookings, and poor time management.

The project primarily aims to create a python CRUD application that allows user to view, add, delete, update their events hence, helping in managing their daily schedules with automatic conflict detection.

PROBLEM STATEMENT

For students and professionals who often manage their busy schedule filled with classes, meetings, and events. Accidental schedule conflicts occur when two events overlap in timing, this leads to missed opportunities or getting double booked. There is a clear need for a simple tool to check for and prevent scheduling conflicts to improve personal time management and priorities.

Functional Requirements

The Project includes:

- The three major functional modules used in the project are user-defined function, if- else conditional statements, Type Conversion, arrays, looping statements.
  - Event Management
    - provides CRUD operations: Add, View, Update, and Delete events.
    - Stores event data using arrays.
  - Conflict Detection
    - Processes time and date input for each event.
    - Uses type conversion (str to int, str to datetime) to compare event timings.
  - User Interaction Module
    - Presents a menu-driven interface for user commands and actions.
    - Implements a logical workflow for the user: menu → input → validation → action → output.

Non-Functional Requirements

- Usability:
  - Anyone with basic Python knowledge should be able to use the application due to its simple, menu-based interface and clear instructions.
- Error Handling:
  - The app checks user input when you enter it, gives it helpful error messages if, for example, the end time comes before the start or if you try to schedule

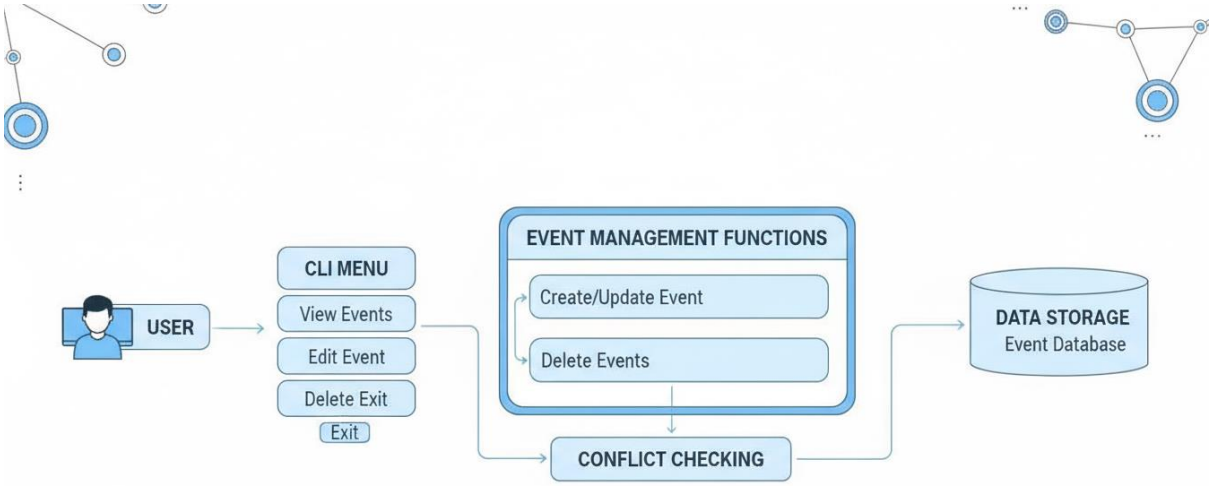
overlapping events.

- Performance:
    - All operations happen instantly, actions like adding, updating, and viewing events are fast—even for a busy schedule.

System Architecture

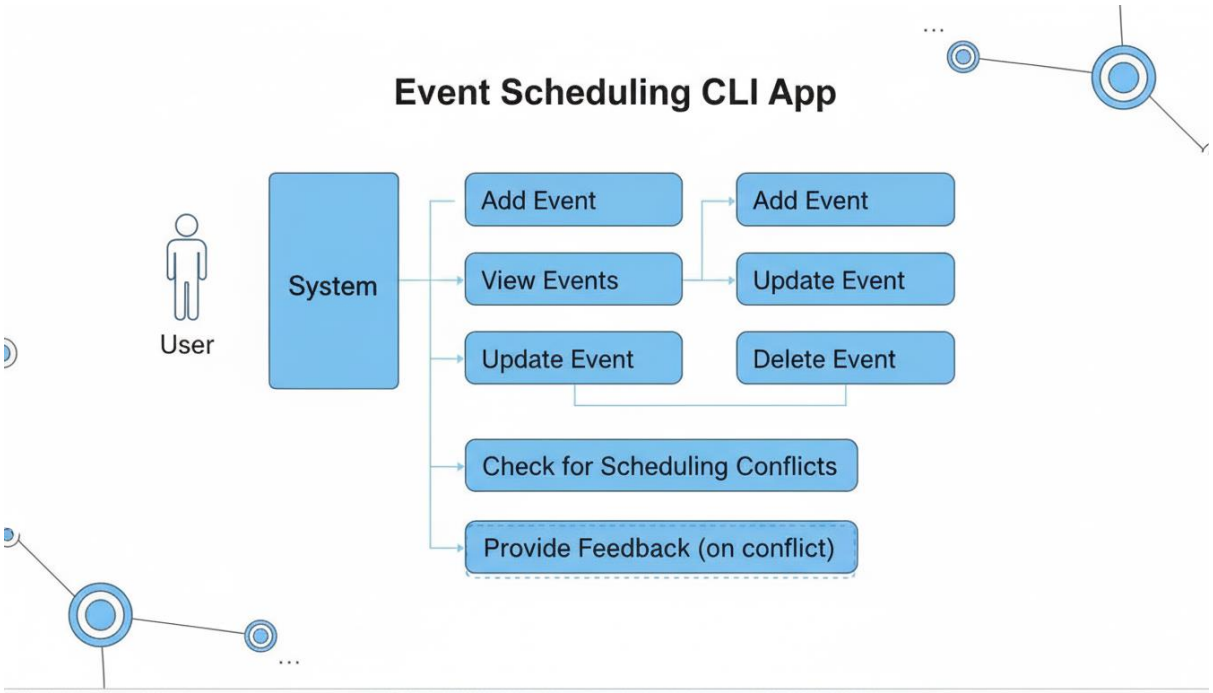
The system architecture consists of a command-line user interface that interacts with core event management modules. All event data is stored in a Python list while the program is running.

The user navigates via a menu to add, view, update, or delete events. Each operation is handled by modular functions. When adding or updating, the system checks for conflicts by comparing event times to existing entries. The architecture is simple and modular, making it easy to understand, use, and extend.

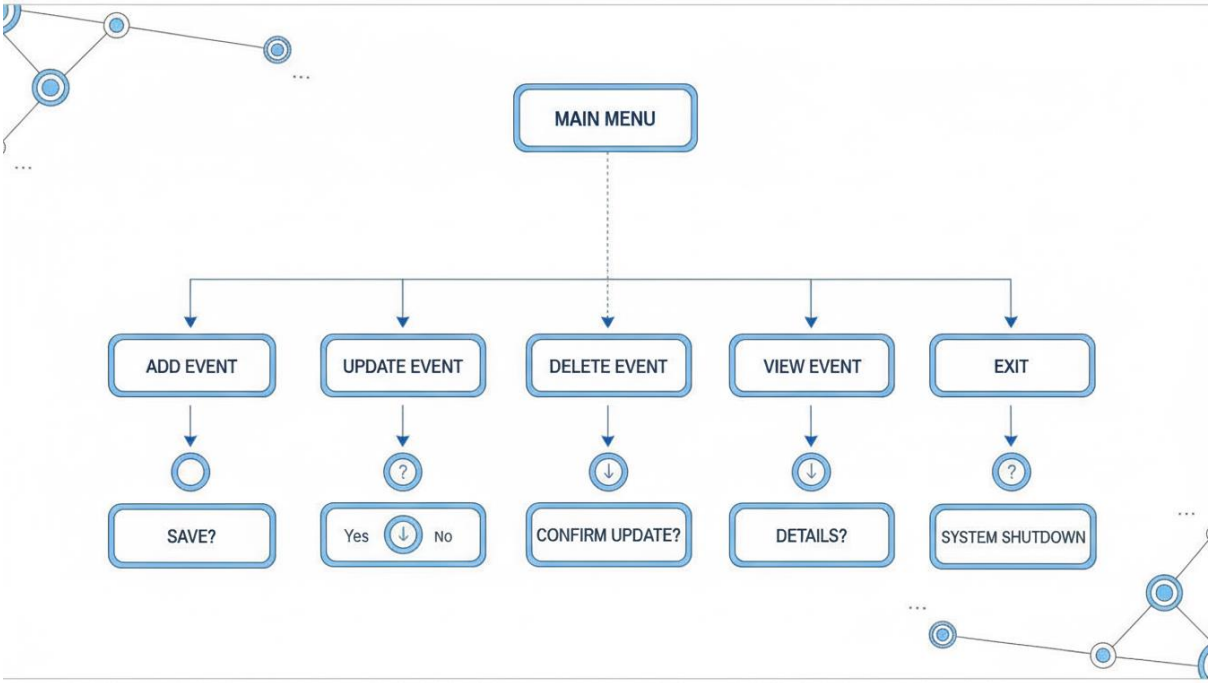


DESIGN DIAGRAMS

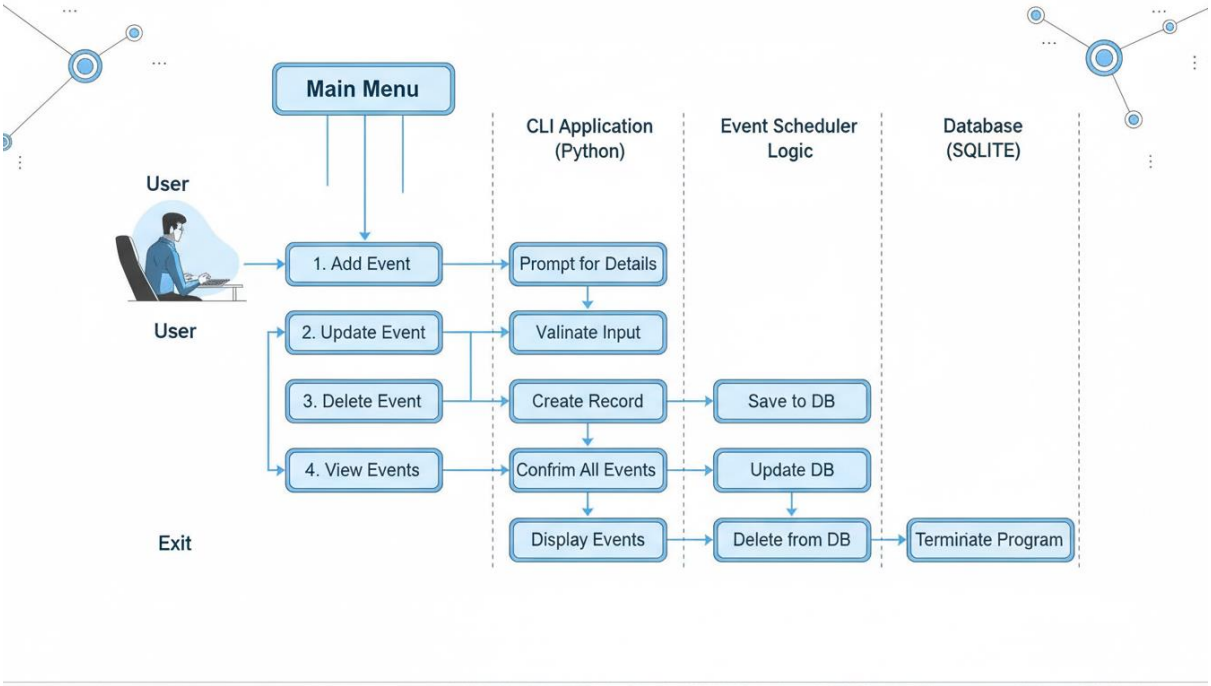
- **Use Case Diagram**



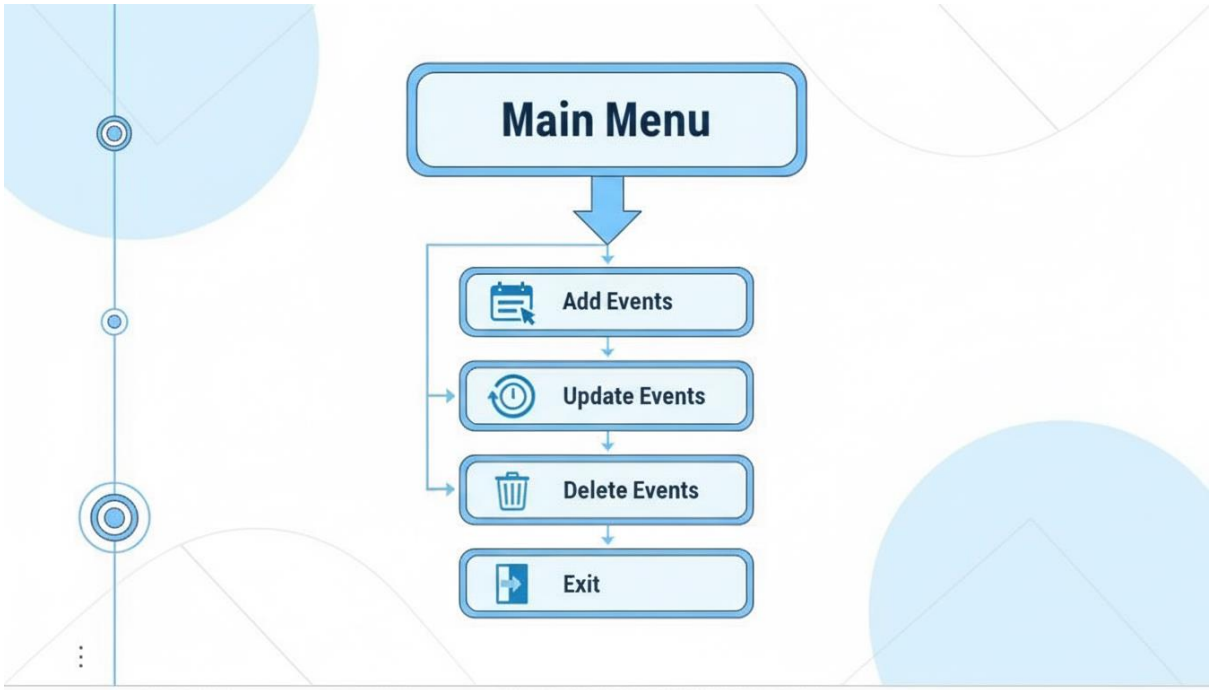
- **Workflow Diagram**



- **Sequence diagram**



- **Component diagram**



DESIGN DECISIONS AND RATIONALES

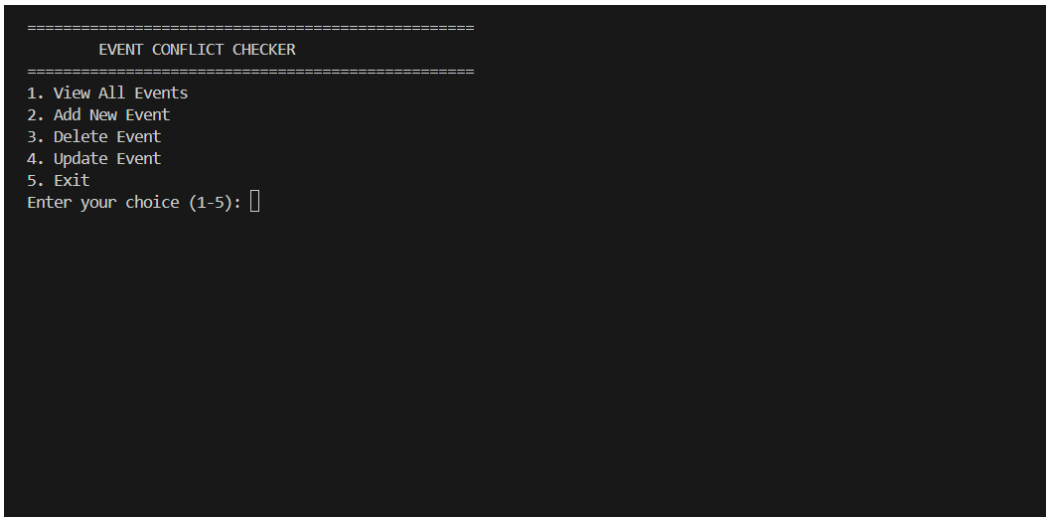
Design Decision	Rationale
Command-line interface (CLI)	Provides a universal, accessible way for students to interact; avoids complexity of GUI/web
In-memory event storage	Keeps project lightweight and fast; sufficient for student/demo scale
List of dictionaries for events	Enables simple, readable CRUD operations and easy iteration/checks
Time conflict checking logic	Prevents overlapping events, directly addressing the scheduling pain point
Use of modular functions	Improves code readability, testing, and maintainability

Implementation details

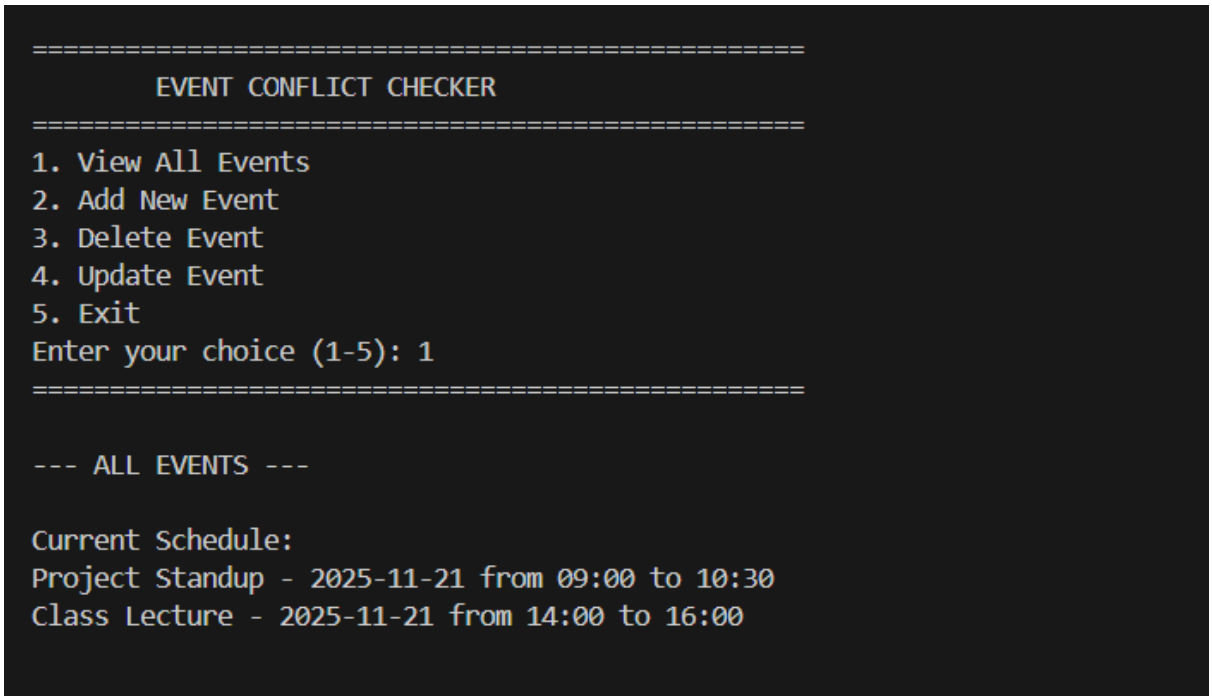
- The program uses a Python list to store all event details (name, date, start, end)
- Main functions include:
    - View schedule
    - Add event
    - Delete event
    - Update event
    - Exit
- A helper function converts time strings to minutes for accurate conflict checks.

Screenshots

1. Main Menu



2. View Schedule



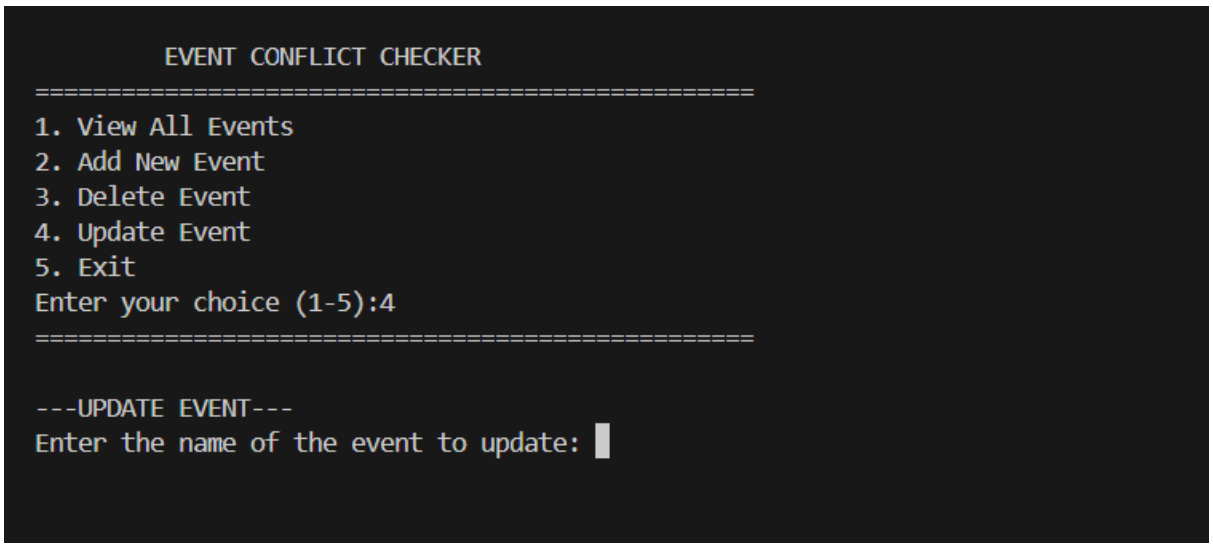
3. Add events

```
==============================================
        EVENT CONFLICT CHECKER
==============================================
1. View All Events
2. Add New Event
3. Delete Event
4. Update Event
5. Exit
Enter your choice (1-5): 2
==============================================

--- ADD NEW EVENT ---
Enter event name: club meeting
Enter event date (YYYY-MM-DD): 2025-12-21
Enter start time (HH:MM): 10:00
Enter end time (HH:MM): 11:00
Event added successfully!

==============================================
        EVENT CONFLICT CHECKER
==============================================
1. View All Events
2. Add New Event
3. Delete Event
4. Update Event
5. Exit
Enter your choice (1-5): 1
==============================================

--- ALL EVENTS ---

Current Schedule:
Project Standup - 2025-11-21 from 09:00 to 10:30
Class Lecture - 2025-11-21 from 14:00 to 16:00
club meeting  - 2025-12-21 from 10:00 to 11:00
```

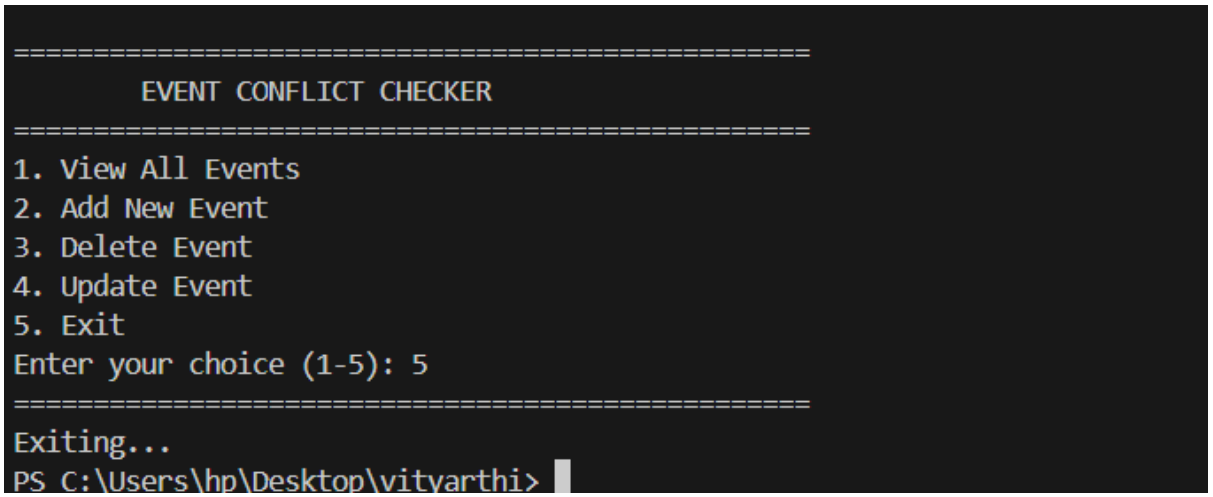
4.Delete Events

```
==============================================
        EVENT CONFLICT CHECKER
==============================================
1. View All Events
2. Add New Event
3. Delete Event
4. Update Event
5. Exit
Enter your choice (1-5): 3
==============================================

--- DELETE EVENT ---
Enter the name of the event to delete: Project Standup
Event 'Project Standup' deleted successfully!

==============================================
        EVENT CONFLICT CHECKER
==============================================
1. View All Events
2. Add New Event
3. Delete Event
4. Update Event
5. Exit
Enter your choice (1-5): 1
==============================================

--- ALL EVENTS ---

Current Schedule:
Class Lecture - 2025-11-21 from 14:00 to 16:00
club meeting  - 2025-12-21 from 10:00 to 11:00
```

5.Update Events



6.Exit



Testing Approach

The testing approach for your Python Event Conflict Checker is manual and interactive.

You test the program by running it in the terminal and using its menu options to add, view, update, and delete events. During these operations, you enter various inputs—valid and invalid—to check the following:

Input validation: The program prompts for event details and checks for invalid times and conflicts.

Conflict detection: It alerts you if you try to add or update an event that overlaps with another.

Feedback messages: It provides clear success or error messages after each action.

Menu navigation: Each function (add, view, delete, update) is accessed through a numbered menu and tested step by step.

Challenges Faced

Challenges included implementing robust input validation for dates and times, crafting accurate logic for detecting overlapping events, and ensuring clear menu navigation with helpful user feedback. Maintaining modular, reusable code while keeping the interface intuitive and error-free required careful planning and iterative refinement throughout development.

Learnings & Key Takeaways

This project strengthened understanding of Python fundamentals, especially data structures, functions, and user input handling.

It reinforced the importance of input validation, clear user feedback, and conflict-checking logic. Effective testing were key to producing a reliable, easy-to-use CLI application

Future Enhancements

Future improvements could include saving schedules to a file for persistence, adding a graphical user interface, importing calendar data, and supporting recurring events. Expanding features for multi-user access, notifications, or integration with Google Calendar would make the tool even more practical and versatile.

References

1. https://www.geeksforgeeks.org/
2. https://stackoverflow.com/
3. https://algo.monster/